
Real-Time Anxiety Detection System for Programmers.

By

Ahmad Hasan

ID: 011221435

Md. Ashiquzzaman Khan

ID: 011221080

Rahat Ahmed Shafi

ID: 011221201

Shariful Islam

ID: 011221078

Pulok Sikdar

ID: 011221167

Md. Mahmudul Hasan

ID: 011221079

Supervised by Dr. Suman Ahmmed



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNITED INTERNATIONAL UNIVERSITY

Abstract

This abstract discusses the essential development and implementation of a real-time anxiety detection system within the cognitively demanding context of software programming. As the field of software engineering continues to evolve and play an increasingly integral role in modern industry, the mental well-being and affective conditions of developers gain prominence. This report highlights the key behavioral features that this proposed CodeBlocks plugin considers, encompassing aspects such as monitoring keystroke dynamics, tracking compile failures, analyzing idle time, and evaluating the frequency of undo/redo operations. Additionally, the abstract emphasizes the significance of utilizing machine learning models and the role of an automated intervention engine in fostering adaptive, context-sensitive support—such as relaxation reminders and workload pacing—for the programming community. By examining the behavioral challenges and real-time intervention opportunities within this specific context, this abstract aims to encourage the adoption of emotionally intelligent development environments, ultimately contributing to lowered developer anxiety, improved productivity, and positive mental well-being.

Acknowledgements

This work would have not been possible without the input and support of many people over the last two trimesters. We would like to express our gratitude to everyone who contributed to it in some way or other. First, we would like to thank our academic advisor, Dr. Suman Ahmmed, for his invaluable guidance, expertise, and continuous encouragement throughout the development of this real-time anxiety detection system. Our sincere gratitude goes to the Department of Computer Science and Engineering at United International University for providing the necessary resources and academic environment to conduct this research. We are also thankful to the 30 university students who willingly participated in our data collection process during their C programming examination; their contribution was essential for training and evaluating our machine learning model. Last but not the least, we owe to our family including our parents for their unconditional love and immense emotional support

Table of Contents

Table of Contents	iv
List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Project Overview	1
1.2 Motivation	1
1.3 Objectives	2
1.4 Methodology	2
1.4.1 System Development	3
1.4.2 Data Collection	3
1.4.3 Feature Engineering	3
1.4.4 Model Training	3
1.4.5 Evaluation	4
1.5 Project Outcome	4
1.6 Organization of the Report	4
2 Background	6
2.1 Preliminaries	6
2.2 Literature Review	6
2.2.1 Similar Applications	6
2.2.2 Related Research	7
2.3 Gap Analysis	8
2.4 Summary	8
3 Project Design	9
3.1 Requirement Analysis	9
3.1.1 Functional and Nonfunctional Requirements	9
3.1.2 Context Diagram	10
3.1.3 Data Flow Diagram	10
3.1.4 UI Design	11

4	Implementation and Results	13
4.1	System Development and Environment Setup	13
4.2	Data Collection	13
4.3	Testing and Evaluation	14
4.4	Results and Discussion	14
4.5	Summary	14
5	Standards and Design Constraints	15
5.1	Compliance with the Standards	15
5.1.1	Software Standards	15
5.1.2	Hardware Standards	15
5.1.3	Communication Standards	15
5.2	Design Constraints	16
5.2.1	Economic Constraint	16
5.2.2	Environmental Constraint	16
5.2.3	Ethical Constraint	16
5.2.4	Health and Safety Constraint	16
5.2.5	Social Constraint	16
5.2.6	Sustainability	16
5.3	Cost Analysis	16
5.4	Complex Engineering Problem	17
5.4.1	Complex Problem Solving	17
5.4.2	Engineering Activities	17
5.5	Summary	18
6	Conclusion	19
6.1	Limitation	19
6.2	Future Work	20
	References	21

List of Figures

1.1	Methodology and system workflow for anxiety detection system.	3
3.1	Context Diagram of the Proposed System	10
3.2	Data Flow Diagram of the Proposed System	11
3.3	Proposed User Interface Design for the Extension	12

List of Tables

2.1	Gap Analysis of Reviewed Literature	8
5.1	Mapping with complex problem solving.	17
5.2	Mapping with complex engineering activities.	18

Chapter 1

Introduction

This chapter presents the project’s background, core objectives, proposed methodology, and expected outcomes, establishing the foundation for the real-time anxiety detection system.

1.1 Project Overview

The proposed project is the design and implementation of an anxiety detector and intervention program in real-time to programmers as an plugin of CodeBlocks IDE. Measures of coding behavior that are passively gathered by the system include keystroke behavior, compile failures, idle time, and the frequency of undo/redo [21, 30]. A machine learning model is used to make a conclusion on the level of anxiety based on the features [4, 19]. Depending on the detected state, the plugin interferes in a timely manner as some reassuring reminders, adaptive feedback messages or workload pacing messages [29]. Consequently, anxiety levels are lowered, productivity is improved and in general, the mental well-being of the programmers is elevated [9].

1.2 Motivation

Cognitively challenging work, programming is accompanied by the unwanted side effects of both anxiety and frustration especially to novices [15, 20]. Constant worry may impair problem solving capacity, decrease debugging performance and also influence negatively on learning performance [3]. Although modern IDEs are created to allow productivity and accuracy, they do not generally address the affective conditions of users like stress or anxiety and which also play a substantial role in programming performance [1]. There is a rising demand of lightweight and IDE-based tools and real-time detection and mitigation of anxiety [12]. The tools will tend to be beneficial to both students and professionals in terms of better emotional control and motivation and general coding experience.

1.3 Objectives

The large objective of the given project is to create an intelligent, real-time anxiety detection and intervention tool in the form of a non-intrusive CodeBlocks plugin. The tool will be kept in a state of observation and monitoring as the coding behavior of programmers; i.e., the keystroke dynamics, the error rate of compiling, idle time, undo/redo operations, etc., will be monitored to conclude about the level of anxiety that a machine learning model can have [2].

The specific objectives of this project are to:

- Have a non-intrusive real time anxiety tracking system in the form of a CodeBlocks plugin fitted into the coding habit as part and parcel.
- The behavior coding measures to be collected and compared include: keystroke speed, error rate, idle time, and the frequency of undo/redo behavior used to identify the patterns of anxiety behavior [25].
- The task is to train and develop a machine learning model that with real-time behavior patterns will forecast levels of anxiety [4].
- Use context-dependent intervention engine that provides just-in-time support as soothing messages, on-the-fly code completion suggestions or workload planning suggestions [29].
- Minimize interference with the coding process of the user, keep productivity and affective well-being in the center [10].
- The comparison of the performance of the system with self-report measures (anxiety ratings on questionnaires) and objective measures (error rate, time taken to complete a task, quality of code).
- Guarantee of a high level of ethical standards through the use of apparent data collection, user authorization, anonymization, and local/on-device processing to the level possible.

1.4 Methodology

The project workflow is divided into distinct phases: system development, data collection, feature engineering, model training, and evaluation. As illustrated in Figure 1.1, the system development phase acts as the first step that lays down the foundation for all subsequent phases. The workflow proceeds with data collection and feature engineering, followed by model training and intervention evaluation. These phases combine to form an end-to-end system that can be used for deploying an anxiety detection system for programmers.

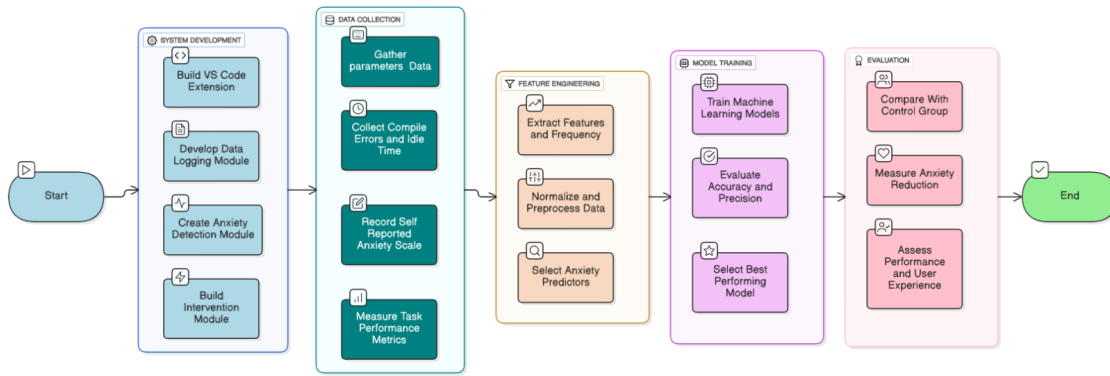


Figure 1.1: Methodology and system workflow for anxiety detection system.

1.4.1 System Development

This initial phase focuses on the core engineering of the tool. It involves building the VS Code extension, which serves as the primary environment for the programmer. Within this extension, three distinct modules are developed: a data logging module to capture real-time metrics seamlessly in the background, an anxiety detection module to process these behavioral signals, and an intervention module to deliver timely support.

1.4.2 Data Collection

Once the core system architecture is developed, quantitative and qualitative data are gathered. This includes collecting general parameter data, tracking the frequency of compile errors, and monitoring developer idle time. To establish a reliable ground truth for the algorithms, self-reported anxiety scales are recorded from the users alongside objective measurements of their task performance metrics.

1.4.3 Feature Engineering

The raw behavioral data collected in the previous phase is refined to optimize the predictive models. This step involves extracting relevant features and calculating their frequencies (such as keystroke dynamics or undo/redo rates). The data is then normalized and preprocessed to ensure consistency, followed by the statistical selection of the strongest anxiety predictors.

1.4.4 Model Training

In this phase, various machine learning models are trained using the engineered features to recognize patterns associated with high cognitive load and stress. The models are rigorously evaluated based on their accuracy, precision, and recall. Ultimately, the best-performing model is selected to drive the real-time detection engine.

1.4.5 Evaluation

The final phase measures the practical impact and effectiveness of the proposed system. The workflow includes comparing the outcomes of developers using the active intervention system against a control group. The system's success is determined by measuring the actual reduction in user anxiety and conducting a comprehensive assessment of overall task performance and user experience.

1.5 Project Outcome

The outcome of the project is the development and evaluation of a mental health-aware extension designed for the real-time detection and addressing of anxiety in programmers [1]. The extension is equipped with the ability to detect anxiety in real-time through the use of machine learning techniques [2, 28]. The following are the key results obtained from the project, which highlight the major achievement of the project:

- **Working Prototype Developed:** The project developed a working extension for the IDE application, which operates in a smooth and seamless manner.
- **Machine Learning-Based Anxiety Detection:** The project was able to develop a working extension for the application that used a machine learning model to classify different anxiety states.
- **Real-Time Feedback and Monitoring:** The application developed has the capability for real-time monitoring of the user in order to identify the anxiety state.
- **Integrated Intervention Mechanism:** The developed extension for the application was able to provide immediate feedback upon the identification of the anxiety state [29].
- **Empirical Evaluation and Influence:** The application was subject to a series of experiments and found to have a positive influence on programming performance and the emotional health of the user.
- **Advancements in Mindfulness-Informed Development Tools:** The project is an example of a trend in emotionally intelligent programming tools [1].

1.6 Organization of the Report

This report is organized into the following chapters:

- **Chapter 1:** Presents background, motivation, and project objective.
- **Chapter 2:** Reviews existing work on anxiety detection, programming behavior, and intervention systems.

- **Chapter 3:** Describes data collection, feature engineering, model training, and evaluation methods.
- **Chapter 4:** Implements the development of the extension and integration of machine learning models.
- **Chapter 5:** Presents model performance, detection accuracy, and intervention efficacy.
- **Chapter 6:** Summarizes key contributions and suggests future research directions.

Chapter 2

Background

This chapter provides the necessary background knowledge to understand the rest of the report, reviewing existing applications, related research, and identifying the research gap this project aims to fulfill.

2.1 Preliminaries

Programming is a very intellectual activity that may become stressful and anxiety causing to the students and the practitioners. It is not only that performance is influenced by prolonged anxiety, but also that motivation can be lost, and time spent debugging may be increased, and even burnout may be experienced. The popular algorithm of supporting programmers, such as the static tutoring systems, debugging aides or code correctness checkers, are more technical than emotional health. Recent advances in machine learning, signal processing of physiological signals, and integrated development environments (IDEs), have indicated that there is the possibility of quantifying emotional states in real time [12, 18]. It is now possible to detect and process facial, biometric, or behavioral measurements to identify the symptoms of anxiety or stress in real-time (e.g., keystroke, mouse motion). These features can be incorporated into the general development environments like Visual Studio Code to enable intervention in the moment and with knowledge without interfering with the programming activity.

2.2 Literature Review

2.2.1 Similar Applications

- **EmotionStream (Personal Informatics):** The study "I spent 14 hours debugging just one assignment" [1] details a Personal Informatics (PI) tool providing insights into emotional states post-session. While relevant, this tool is designed for reflection, not real-time, automated intervention, representing a key gap this project fills.

- **Developer Mental Health Toolkit (VS Code Extension):** This extension tracks the coding activity and reminds about micro-breaks, wellness tips, and stress-reducing exercises in the Visual Studio Code itself. It however does not have automated anxiety detection and is based on fixed reminders and not the adaptive interventions.
- **Nafas - Breathing Gymnastics Application:** Nafas is a command line application (CLI) which users of the computer use to provide breathing exercises in the terminal. Nafas gives coders a break through simple exercises that are timed without requiring one to move out of the workstations. Although it is very effective in the management of stress, Nafas lacks automatic detection of anxiety and does not interfere in real-time when stress reaches its peak.
- **WakaTime (VS Code Extension):** WakaTime is an open source VS Code extension used to record coding activity and display real time analysis of the coding time. However, while self-monitoring is supported by the plugin, it lacks the ability to monitor the state of anxiety in real-time and be able to actively intervene.

2.2.2 Related Research

- **Programming Anxiety in Education:** Systematic reviews confirm that programming anxiety remains a major challenge in higher education, severely affecting persistence and performance [3, 15]. However, most studies used surveys and interviews, with little focus on real-time detection.
- **AI-Assisted Interventions:** AI copilots are shown to reduce general programming anxiety [3, 16], but the challenge remains: these interventions are typically not driven by automatic detection but by the availability of support.
- **Physiological Stress Detection:** Some studies on multimodal wearable sensing show that physiological features like HRV and EDA are effective for anxiety detection [17, 18]. However, these features tend to be noisy in uncontrolled environments, thus the need to combine physiological and contextual IDE data.
- **Wearable Stress Detection Reviews:** Various reviews emphasize the limitations of real-time usage and user acceptance of wearable devices [2, 6], suggesting the use of simple techniques such as keystroke and eye-tracking analysis.
- **Programming Anxiety Prediction Models:** Research carried out using Machine Learning (ML) for detection [19] and prediction [4] confirms the basic premise of ML. However, these models are based on coarse-grained features, which do not accurately represent the anxiety of programming.
- **Developer Well-Being and Industry Context:** Studies establish the fact that a vast number of professional software developers suffer from stress, burnout, and

imposter syndrome [1, 10].

- **Multi-Modal Fusion in Programming Contexts:** Multi-Modal Fusion in Programming Contexts [13, 14], keystrokes [11], and behavioral parameters [12] has shown that the fusion of these modalities enables more accurate detection of struggles than the use of single modalities.

2.3 Gap Analysis

The real-time anxiety detection systems have solid background in the research that has been done on the fields of programming education, mental health monitoring and machine learning. Yet, there is no solution so far that will integrate real-time anxiety detection and intervention into an IDE.

Paper Focus	Current Limitation	Research Gap Addressed
Personal Informatics [1]	Relies on self-reported data; lacks real-time detection during coding process.	Real-Time Detection in coding environment.
Programming Anxiety in Higher Education [3]	Lack of Standardized/Validated Assessment Tools	Real-time, context-aware intervention.
Human Stress Detection [2]	A review paper; does not present a primary study in programming domain.	Specific ML Application for anxiety in the programming domain.
Code Review Anxiety [10]	Focuses on code review (post-coding activity); lacks real-time coding focus.	Real-Time Detection during Coding.
Eye Tracking Data [13]	Lacks the design and evaluation of an intervention system.	Full-Cycle Intervention System (Detection + Mitigation).

Table 2.1: Gap Analysis of Reviewed Literature

2.4 Summary

The support of AI tools can make programming less stressful and more motivated, as it was proven by previous researches. It is possible to identify emotional states using behavioral data (keystroke, coding logs, etc.). However, a solution that will introduce the real-time detection and intervention of anxiety and directly integrate into an IDE has not been found to date. This gap occupation can lead to low-cost, non-invasive, and efficient gap occupation to support the well being of programmers, the learning process, and the risk of burnout.

Chapter 3

Project Design

Every chapter should start with 1-2 sentences on the outline of the chapter. This chapter highlights the requirement analysis, high-level and detailed design of the system, including context and data flow diagrams, as well as the project plan and task allocation.

3.1 Requirement Analysis

3.1.1 Functional and Nonfunctional Requirements

Functional Requirements

- The system shall be able to detect programmer anxiety in real-time using behavioral and contextual information.
- The system shall be able to provide adaptive interventions within the environment of VS Code.
- The extension shall be able to log anxiety-related events for further analysis.
- The system shall be able to ensure the privacy of the user.

Nonfunctional Requirements

- **Performance:** The system shall be able to instantly detect threats and execute countermeasures at the fastest possible speed.
- **Usability:** The extension shall be able to occupy a minimal space for installation and shall not interfere with the coding process of the developer.
- **Scalability:** The system shall be able to support multiple programming courses and different user groups simultaneously.
- **Security:** The information gathered shall be private and anonymous.

3.1.2 Context Diagram

In the context diagram, we show the high-level interaction of the proposed anxiety detection system with the outside world. At the core of the system, we have the VS Code Extension that collects data from the Programmer User (File Keystrokes, File Active m/s, Keystrokes Total, Active m/s Total, Idle m/s Total, Avg Key m/s, Undo Count, Redo Count, Compile Attempts, JS Errors, eye tracking data such as blink rate and fixation patterns), as well as data from the Coding Environment IDE in the form of environment events and contextual data.

The diagram illustrates a closed-loop feedback system where the extension acts as the primary data aggregator. All the data collected by the extension is then forwarded to the Anxiety Detection Engine, which analyzes the data and provides Analysis Results. Finally, the Anxiety Alert and Recommendation System provides the results back to the extension, while the collected data and events are stored in Database Logs for future model refinement.

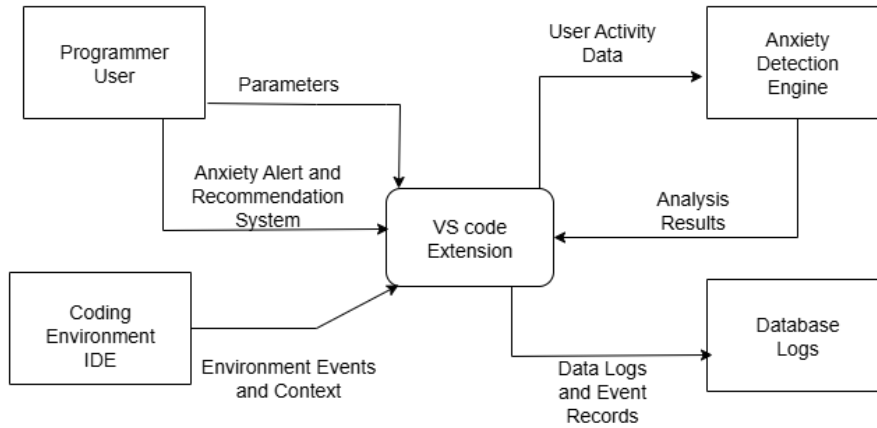


Figure 3.1: Context Diagram of the Proposed System

3.1.3 Data Flow Diagram

The proposed systems for real-time anxiety detection are well grounded in the research done in the fields of programming education, mental health monitoring, and machine learning. Previous research has shown that programming can be more motivated and less stressful with the help of AI tools [3, 16]. Emotional states can be detected based on behavioral data such as keystrokes [21, 30]. More accurate results are achieved by using automated sensing together with self-reporting [1]. Yet, there is no solution so far that will integrate real-time anxiety detection and intervention into an IDE. This gap occupation may lead to the creation of non-invasive, cost-effective, and efficient systems to aid programmer well-being, the learning process, and prevent burnout.

As shown in Figure 3.2, the system architecture is decomposed into specific functional modules. The proposed system would capture Programmer activity from both the Pro-

grammar and IDE environment through the Capture Input module. User inputs would be gathered through Keystrokes, activity logs, and behavioral indicators. The system would utilize the Detect Anxiety module to analyze user inputs which would enable it to detect anxiety. It would utilize the Intervene module to provide Feedback and Notifications to users, as well as log all intervention data. It would utilize the Log Data module to analyze data gathered from the system's results as well as intervention records to log data in Logs for system records and History for long-term tracking for future analysis.

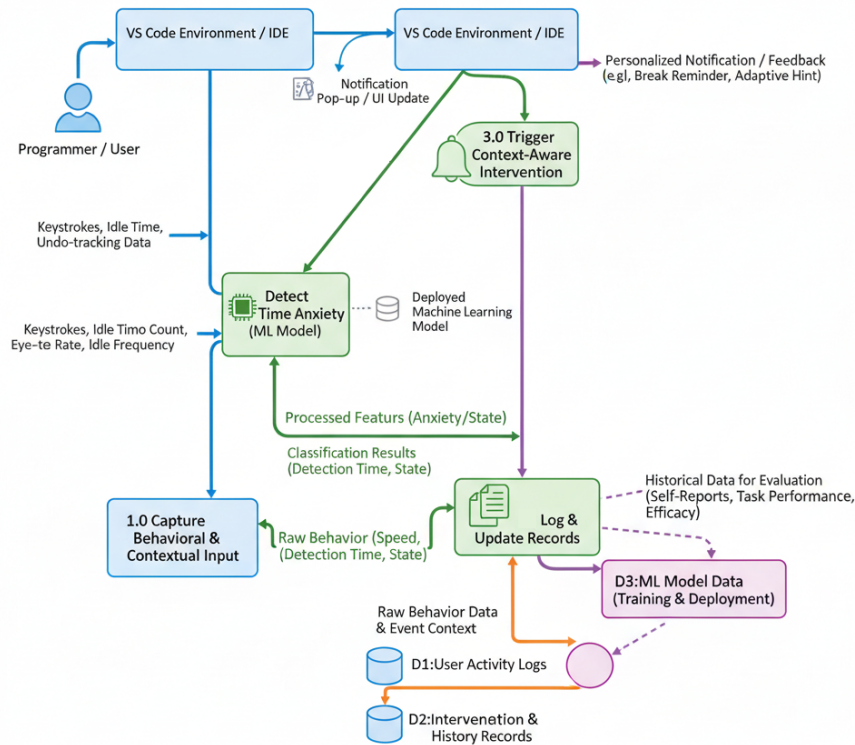


Figure 3.2: Data Flow Diagram of the Proposed System

3.1.4 UI Design

The User Interface design focuses on non-intrusive awareness and accessibility. As shown in Figure 3.3, the "Anxiety Copilot" side panel provides a high-level view of the user's current state, showing real-time anxiety levels and focus trends. The interface includes an Intervention History log to help developers reflect on past stress triggers and a Settings menu to toggle features like eye-tracking and real-time monitoring. A key component of the UI is the notification toast, which appears as a gentle suggestion for a mindful break or a breathing exercise when high anxiety is detected.

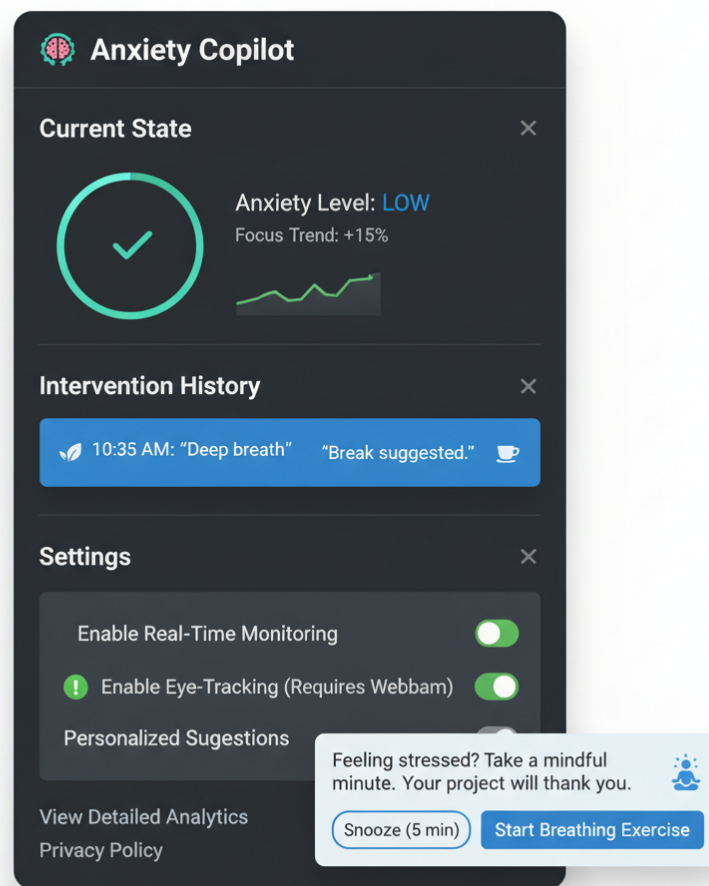


Figure 3.3: Proposed User Interface Design for the Extension

Chapter 4

Implementation and Results

This chapter provides a detailed account of the technical implementation, starting from the system development and environment setup to the final evaluation of the anxiety detection models. It discusses the specific software components, data collection methodologies, and the comparative performance of various machine learning algorithms.

4.1 System Development and Environment Setup

The methodology for the project started with the System Development phase. This, according to the system architecture, required the construction of the IDE extension, the development of the Data Logging Module, the creation of the Anxiety Detection Module, and the construction of the Intervention Module.

To achieve the system architecture, the real-time system was developed as a smart, non-intrusive plugin for the Code::Blocks IDE. The plugin works in the background continuously through three main subsystems:

- **Data Logging Module (EventManager):** It continuously observes the patterns of keystrokes and compilations in real-time to extract useful behavioral features.
- **Anxiety Detection Module (PythonBridge):** A backend service that receives data features at regular intervals and uses the pre-trained machine learning model (`best_anxiety_model.pkl`) to make an anxiety state prediction and calculate the confidence level.
- **Intervention Module (InterventionManager):** Evaluates if the developer needs a prompt based on certain behavioral triggers and an anxiety threshold.

4.2 Data Collection

The second phase, Data Collection, aimed to obtain parameter data, collect compile errors and idle time, record self-reported anxiety scales, and measure task performance metrics.

During this phase, data collection for a group of 30 to 32 students was carried out using a local Python-based collector script (`START_CLEAN_COLLECTOR.bat`).

During the sessions, the following observations were made:

- Repeatedly encountering compile errors (64.5%), long debugging sessions (58.1%), and nearing deadlines (58.1%) were the main situations that triggered anxiety.
- 71.9% of students reported frustration and fear of failure when they got stuck, while 68.8% reported feelings of panic or stress.
- Behaviorally, 75% of students who were anxious tended to repeatedly undo/redo code, 68.8% stayed idle for long periods, and 50% took frequent breaks.

4.3 Testing and Evaluation

The Feature Engineering phase involved the systematic extraction of feature frequencies and normalization. Some of the extracted features included Typing Velocity, Keystroke Variance, Backspace Rate, and the Rate of Error Discovery (RED Metric). The analysis identified the top behavioral predictors of programmer anxiety; the most significant factors were the `BACKSPACE_RATE`, `COMPILE_ERROR` frequency, and `UNDO_REDO_ATTEMPT` counts.

The Model Training phase involved the evaluation of three models to identify the best fit for the project requirements:

- **Support Vector Machine (SVM) Model:** Achieved an accuracy of 92.18%.
- **XG Boost Model:** Achieved an accuracy of 92.11%.
- **Random Forest Model:** Achieved an accuracy of 91.5%.

4.4 Results and Discussion

The Support Vector Machine (SVM) was chosen as the final model based on its balanced performance for high and low anxiety classification. The evaluation demonstrates that real-time behavioral monitoring can successfully identify distinct anxiety states. Distribution analysis revealed that programmers spent approximately 35.56% of session time in a high anxiety state, while medium anxiety accounted for 50%.

4.5 Summary

In summary, the implementation successfully integrated a high-accuracy SVM model into the Code::Blocks environment. The results confirm that behavioral metrics like backspace usage and compile errors are reliable indicators of programmer stress. This implementation provides a scalable framework for improving developer well-being through real-time, mindfulness-informed feedback.

Chapter 5

Standards and Design Constraints

This chapter outlines the engineering standards and design constraints that guided the development of the real-time anxiety detection system. It provides a detailed justification for the chosen standards, an analysis of costs and revenue models, and a formal mapping of the project to complex engineering problem categories.

5.1 Compliance with the Standards

The system is developed in accordance with international software, hardware, and communication standards to ensure reliability, ethical alignment, and interoperability.

5.1.1 Software Standards

The project adheres to IEEE 830-1998 for requirements and IEEE 7000-2021 for ethical AI alignment. These provide a technical framework for handling sensitive emotional data while ensuring informed consent and safe data handling.

5.1.2 Hardware Standards

Compatibility is ensured through the USB Video Class (UVC) standard for webcams, while ISO 9241-210:2019 ergonomics standards ensure the hardware-software interaction is non-intrusive for long-term use.

5.1.3 Communication Standards

Data transmission is secured via TLS 1.3 and HTTPS. The architecture utilizes a RESTful API to ensure low-latency communication between the IDE extension and the Python-based analytics engine.

5.2 Design Constraints

The system’s design is governed by several critical constraints to ensure its viability and ethical integrity.

5.2.1 Economic Constraint

The system must be affordable for individual developers and students. This constrained the hardware choice to standard webcams instead of high-end biometric sensors.

5.2.2 Environmental Constraint

The system is designed to operate within the Code::Blocks IDE. It must function in various lighting conditions (for eye-tracking) without requiring a specialized laboratory environment.

5.2.3 Ethical Constraint

Strict adherence to privacy is required. No actual source code or identifiable text is stored; only behavioral frequencies (e.g., backspace count) are processed.

5.2.4 Health and Safety Constraint

The intervention engine must not be overly aggressive. If suggestions are too frequent, they could distract the developer, potentially causing more stress or accidental errors in critical code.

5.2.5 Social Constraint

The tool aims to bridge the gap in mental health support for developers, promoting a culture where emotional well-being is integrated into the workflow.

5.2.6 Sustainability

The project leverages open-source libraries (TensorFlow, Scikit-learn), ensuring that the system can be maintained and updated by the community without heavy licensing costs.

5.3 Cost Analysis

The project utilizes a cost-effective architecture. The proposed budget is 50–100 USD, leveraging standard UVC webcams and open-source software, which is significantly more accessible than proprietary biometric alternatives costing over 1,000 USD.

5.4 Complex Engineering Problem

The project is categorized as a complex engineering problem as it requires the integration of Machine Learning, Human-Computer Interaction, and Ethical AI standards.

5.4.1 Complex Problem Solving

In this section, we map the project attributes to the complex problem-solving categories (P1–P7) as shown in Table 5.1.

Table 5.1: Mapping with complex problem solving.

P1	P2	P3	P4	P5	P6	P7
Depth of Knowledge	Range of Conflicting Requirements	Depth of Analysis	Familiarity of Issues	Extent of Applicable Codes	Extent of Stakeholder Involvement	Inter-dependence
✓	✓	✓	×	✓	✓	✓

The following points justify the selection of attributes in Table 5.1:

- **P1 (Depth of Knowledge):** The project requires a deep knowledge profile involving advanced machine learning (K4) and engineering design (K5). It draws from multiple disciplines, including psychological modeling and low-level IDE API integration.
- **P2 (Conflicting Requirements):** There is a significant trade-off between the depth of data collection (for accuracy) and the need for user privacy. Similarly, real-time monitoring must be high-frequency without causing CPU lag in the IDE.
- **P3 (Depth of Analysis):** The system requires original modeling of behavioral patterns (keystroke dynamics) to detect an abstract emotional state like anxiety, which cannot be solved using routine data analysis.
- **P5 (Applicable Codes):** The design is strictly constrained by a wide range of international standards, including ISO 27001 for security and IEEE 7000 for AI ethics.
- **P7 (Inter-dependence):** The system consists of highly interdependent subsystems. A failure in the Sensor Layer (C++ plugin) directly results in an inability for the Intelligence Layer (Python ML) to function.

5.4.2 Engineering Activities

The project involves complex engineering activities (A1–A4) characterized by multidisciplinary interaction and high societal impact, as summarized in Table 5.2.

The attributes marked in Table 5.2 are described as follows:

Table 5.2: Mapping with complex engineering activities.

A1	A2	A3	A4	A5
Range of re-sources	Level of Interac-tion	Innovation	Consequences for society and environment	Familiarity
✓	✓	✓	✓	×

- **A1 (Range of Resources):** The activity involves a diverse range of resources, including machine learning infrastructure, real-time data streaming protocols, hardware webcams, and human test participants.
- **A2 (Level of Interaction):** There is a strong interdisciplinary coupling between UX/UI design (for non-intrusive interventions) and backend algorithmic logic.
- **A3 (Innovation):** The system introduces a novel real-time integration of behavioral monitoring and just-in-time micro-interventions directly within the development workflow, moving beyond standard post-session analytics.
- **A4 (Consequences):** The system has direct consequences for workplace well-being and productivity. It also carries significant ethical implications regarding how personal emotional data is handled in a professional setting.

5.5 Summary

This chapter established the technical and ethical boundaries of the project. By adhering to international standards and addressing complex engineering constraints, the system ensures a reliable and professional approach to programmer well-being.

Chapter 6

Conclusion

6.1 Limitation

Although the suggested Anxiety Detection Plugin was shown to be very accurate (92.18% via SVM) in detecting programmer stress due to behavioral measures, a number of limitations should be considered:

- **Demographic Constraints:** The model was developed and tested on a relatively small sample of 30 to 32 university students when they were being assessed in an exam. This particular stressful environment and demographic, may not adequately capture the behavioral trends, resiliency, stressors, and daily life of the professional software engineer working in enterprise environments.
- **Platform Dependency:** In the existing implementation, it is strongly bound to the Code::Blocks IDE. Although this is suitable in the desired C/C++ academic setting, it restricts the direct suitability of the system to current diverse development processes that utilize editors such as VS Code or IntelliJ.
- **Subjective Ground Truth:** The use of self-reported pre- and post-surveys (as is the case of STAI) to determine ground-truth anxiety labels creates subjective bias. The capacity of the participants to correctly measure their own mental condition may be inconsistent and may serve as noise to training data.
- **Absence of Physiological Data:** The system does not require anything more than mechanical interactions (keystrokes, WPM, compile errors). Although this makes it non-intrusive, it does not offer the conclusive biological indication of stress (e.g. heart rate variability, galvanic skin response) that physiological sensors might offer to confirm the behavioral measures.

6.2 Future Work

Future development and studies in this research ought to take the following avenues in order to build on the foundation set on this research: :

- **Cross-Platform Integration:** The Data logging and Intervention modules architecture must be abstracted to form a universally compatible extension architecture and be deployed to the state-of-the-art IDEs of the era such as Visual Studio Code, JetBrains IDEs, and Eclipse with ease.
- **Large-Scale, Diverse Data Collection:** Future research should expand the data collection procedure to encompass a wider range of developers between novice and senior engineers on long term, real-life projects. This will assist in generalizing the machine learning model and in avoiding overfitting to the situation of academic exams.
- **Multimodal Anxiety Detection:** The predictive model might be improved significantly, when fused with non-invasive physiological monitoring of the current behavior metrics. With the use of the standard webcams available to conduct real-time eye-tracking, monitor the blink rates, or analyzing the facial expression, the percentage of detection might improve significantly.
- **Generative AI Interventions:** The present Intervention Manager is based on a list of hints and break recommendations. As a future work, Large Language Models (LLM) might be incorporated to examine the syntax of repeated instances of COMPILE ERROR and produce highly personalized, context-specific code corrections to take action to reduce the instances of such errors.

References

- [1] A. Chandrasekaran, L. Bielicke, D. Shah, H. Janakiraman, and M. L. Mauriello, “I spent 14 hours debugging just one assignment’: Toward Computer-Mediated Personal Informatics for Computer Science Student Mental Health,” in *Proc. CHI Conf. on Human Factors in Computing Systems (CHI ’25)*, 2025, doi: 10.1145/3706598.3713269.
- [2] A. Javed, A. Mahmood, and R. Kousar, “Human Stress Detection Using Machine Learning Algorithms: A Review,” *Electronics*, vol. 11, no. 3, p. 389, 2022.
- [3] Y. Li, X. Tan, and S. Kim, “Understanding and Mitigating Programming Anxiety in Higher Education: A Systematic Literature Review,” *STEM Education Journal*, vol. 12, no. 1, p. 537, 2025.
- [4] R. Verma and P. Singh, “Anxiety Prediction and Analysis: A Machine Learning Based Approach,” *ResearchGate Preprint*, 2023.
- [5] D. McDuff and R. El Kaliouby, “Applications of Automated Facial Coding in Media Measurement,” in *Proc. 18th ACM Int. Conf. on Multimodal Interaction*, pp. 115–122, 2016.
- [6] R. Kumar and S. Sharma, “A Survey on Stress Detection Techniques Using Physiological Signals,” 2022.
- [7] I. Bishop et al., “Using Spatial Data to Model Human Stress Factors,” *ISPRS Int. J. Geo-Information*, vol. 11, no. 11, p. 552, 2022.
- [8] J. A. Healey and R. W. Picard, “Detecting Stress During Real-World Driving Tasks Using Physiological Sensors,” in *Human-Computer Interaction*, Springer, pp. 1–10, 2013.
- [9] T. Ahmed, S. Chowdhury, and M. Rahman, “Real-Time Emotional Self-Regulation and Stress Monitoring System,” *ResearchGate Preprint*, 2023.
- [10] A. Brown and L. Davis, “Understanding and Effectively Mitigating Code Review Anxiety,” *ResearchGate Preprint*, 2024.
- [11] J. Nguyen, Y. Wang et al., “Associations Between Smartphone Keystroke Metadata and Mental Health Symptoms in Adolescents,” *ResearchGate Preprint*, 2023.

- [12] R. Picard et al., “Real-Time Monitoring of Behavioural Parameters Related to Psychological Stress,” *IEEE Trans. on Affective Computing*, 2012.
- [13] J. Wu, H. Li, et al., “Stress Detection Using Eye Tracking Data: An Evaluation of Full Parameters,” *ACM Trans. on Computing for Healthcare*, 2023.
- [14] L. Zhang and Y. Wang, “Stress Detection Using Eye Tracking Data: An Evaluation of Full Parameters,” in *Human-Computer Interaction and Beyond*, Springer, 2024.
- [15] X. Wang and M. Lee, “Cognitive Load and Programming Anxiety in Online Learning Environments,” *Journal of Computing in Higher Education*, vol. 32, pp. 265–284, 2020.
- [16] Y. Li and S. Kim, “Understanding and Mitigating Programming Anxiety in Higher Education: A Systematic Literature Review,” *STEM Education Journal*, 2025.
- [17] H. Zhou and Y. Chen, “Physiological Signal-Based Anxiety Detection Using Deep Learning,” in *Proc. CHI Conf. on Human Factors in Computing Systems*, 2024.
- [18] Z. Yin et al., “A Deep Learning Approach for Emotion Recognition Using Physiological Signals,” in *Proc. ACM Int. Conf. on Bioinformatics and Computational Biology*, 2021.
- [19] A. Ahmed, R. Sultana, M. T. R. Ullas, M. Begom, M. M. I. Rahi, and M. A. Alam, “A machine learning approach to detect depression and anxiety using supervised learning,” in *Proc. 2020 IEEE Asia-Pacific Conf. on Computer Science and Data Engineering (CSDE)*, 2020, pp. 1–6, doi: 10.1109/CSDE50874.2020.9411642.
- [20] Z. Atiq and M. C. Loui, “A Qualitative Study of Emotions Experienced by First-year Engineering Students during Programming Tasks,” *ACM Trans. Comput. Educ.*, vol. 22, no. 3, Art. 32, pp. 1–26, 2022, doi: 10.1145/3507696.
- [21] C. Epp, M. Lippold, and R. L. Mandryk, “The Influence of Emotion on Keyboard Typing,” *PLOS ONE*, vol. 10, no. 5, e0129056, 2015.
- [22] J. Zulueta et al., “Predicting Mood Disturbance Severity with Mobile Keystroke Dynamics,” *JMIR Mental Health*, vol. 5, no. 7, e241, 2018.
- [23] J. Zulueta, A. P. Demos, C. Vesel, M. Ross, A. Piscitello, F. Hussain, S. A. Lange-necker, M. McInnis, P. Nelson, K. Ryan, A. Leow, and O. Ajilore, “The Effects of Bipolar Disorder Risk on a Mobile Phone Keystroke Dynamics Based Biomarker of Brain Age,” *Frontiers in Psychiatry*, vol. 12, p. 739022, 2021.
- [24] P. Arean et al., “Are Anxiety Detection Models Generalizable? A Cross-Dataset Study,” *arXiv preprint arXiv:2504.03695*, 2025.
- [25] M. Bunia et al., “Recognizing User Emotion Based on Keystroke Dynamics,” *Procedia Computer Science*, 2024.

-
- [26] T. Yamazaki et al., “A Novel Scale for Anxiety-Related Fixation Instability During Gaze Tasks,” *Scientific Reports*, 2025.
 - [27] A. Calikusu et al., “Pupilometer Efficacy in Monitoring Anxiety During Academic Exams,” *Nature Scientific Reports*, 2025.
 - [28] L. Rahimi et al., “The Role of AI in Identifying Depression and Anxiety: A Comprehensive Review,” *Cureus*, 2024.
 - [29] P. Kaundal et al., “Effectiveness of Just-in-Time Adaptive Interventions for Mental Health Systems,” *BMJ Mental Health*, 2025.
 - [30] W. Zhang and D. Kim, “Keystroke Dynamics: Concepts, Techniques, and Applications,” *ACM Computing Surveys*, 2023.